

Document: The "Operating System" for Organizations.

FSR-OS is a high-performance, real-time workspace designed specifically for university student councils (FSRs) and event-driven organizations. It provides a Notion-like organization experience without data lock-in or privacy violations.

Using a "Bring Your Own Storage" (BYOS) architecture, FSR-OS acts as the real-time processing engine. All data is saved securely and directly to the institution's existing cloud (Sciebo/Nextcloud) via WebDAV, ensuring strict DSGVO (GDPR) compliance.

[TechStack](#)

The "North Star" Principles:

1. **Bring Your Own Storage (BYOS):** We are the "engine," the university is the "vault." We process data in real-time but never own it.
 2. **Real-time or Nothing:** Every interaction—from dragging a Kanban card to editing a meeting protocol—must feel instant and simultaneous.
 3. **Institutional Privacy:** Built for the strict German GDPR landscape. Data lives in NRW, is processed in NRW, and stays under the control of the FSR.
 4. **Zero Configuration:** An FSR should be able to connect their Sciebo and have a functional workspace in under 60 seconds.
-

The Architecture (WebSockets + BYOS)

- **The Engine:** A lightweight Node.js VPS server acts as the middleman. The frontend does not write directly to Sciebo files.
- **The Data Flow:** The server loads the `.fsr/json` file from Sciebo into RAM. Clients connect via WebSockets. Every edit is broadcast instantly to all connected users for a smooth, simultaneous experience.
- **The Save State:** The server periodically writes the synchronized RAM state back to the actual file on the Sciebo cloud.
- *(Note: Earlier concepts using Netlify/Serverless are deprecated. The stack is strictly SvelteKit for the frontend and a stateful Node.js VPS for the backend)*

[Core File Structure](#)

Dual View Modes

- **OS / Backend View:** The internal file directory system. Authenticated members use tools (Kanban, tables) and manage permissions. (Guest Access without Login is possible but needs to be set as "allowed" in Organisation Settings)
 - **Web Preview / Frontend View:** A stylized, public-facing website for the organization. It uses their specific branding (logo, colors) and lists active events, public documents, helper rosters, and Stripe checkout links.
-

Core File Types (The Blocks)

[File Types](#)

Global OS Features

- **Internal File Linking:** Custom routing (`fsrc://[file_id]`) to connect documents to Kanban cards or shifts.
- **File Previews:** Hovering over a linked file displays a miniature preview.
- **Profile Settings:** Custom display names and avatar images for users.
- **Team Labels:** Tagging users (e.g., "Social Media Team") for quick assignment and filtering.
- **Role and Permission Management**

[Accounts Management and User Preferences](#)

- add Dates to every File Type
- Linking variables between files. Also the possibility to call a filename as a already set variable
- global search
- cross-file variables
- global announcements
- settings panels as overlays so the current opened file stays active
- global files
- global topics, files and folders can be assigned to these topics. Topics can have different states like "active", "paused", ...

[Feature Explanation](#)

Phased Rollout Plan

Phase 1: Closed Beta (Concierge Onboarding)

- **Infrastructure:** One central VPS (e.g., a 5€ Netcup VPS 1000). The multi-tenant architecture (`tenant_id`) and subdomain routing (`fsrname.fsr-os.de`) are fully active.
- **The Process:** FSRs cannot sign up themselves. I act as the "concierge." When an FSR wants the tool, I manually add a row to your server's local SQLite database with their desired subdomain and their Sciebo credentials.
- **The Advantage:** I maintain absolute control over who uses the system while it is still in beta. If the server crashes, I only have to restart one machine. I don't have to build complex, secure, self-serve onboarding UI yet.

Phase 2: Public SaaS (Self-Serve Automation)

- **Infrastructure:** The exact same central server (potentially upgraded to more RAM if traffic is high).
- **The Process:** You build the public landing page. FSRs click "Start Free Trial," type in their desired organization name, and a script automatically validates the name, creates the subdomain routing, securely saves their Sciebo App Password to the database, and boots up their workspace.
- **The Upgrade:** You integrate Stripe for automated monthly billing and subscription management.

[Organization Setup Flow](#)

Future Extensibility:

Decentralized Plugin Architecture

Code Sovereignty and Customization

To accommodate organization-specific workflows (e.g., custom Jira integrations, specialized export formats, or unique theme engines) without bloating the core application, the OS supports a decentralized plugin ecosystem. Aligning with the Bring Your Own Storage (BYOS) philosophy, organizations maintain absolute code sovereignty by hosting their own active extensions directly within their WebDAV storage, rather than relying on a centralized marketplace.

1. Storage & Organization (**config.sysfolder/plugins.sysfolder/**)

- **Function:** The physical repository for third-party or custom-built extensions.
- **Data Structure:** Contains the raw, compiled JavaScript (.js) and CSS (.css) files.
- **Implementation:** Because the SvelteKit frontend operates as a flexible client-side shell, it dynamically fetches and executes these files based on the active organization's configuration. If a plugin update breaks a workflow, an admin can simply revert the .js file in their Sciebo folder to immediately restore functionality.

2. The Plugin Registry (**config.sysfolder/plugins.frsys**)

- **Function:** The configuration file dictating which plugins are active and their allowed operational scopes.
- **Data Structure:** A structured map linking specific plugin files to their required RBAC (Role-Based Access Control) permissions (e.g., granting a calendar plugin **files.read** but explicitly denying **files.delete**).
- **Implementation:** During the login sequence, the OS reads this registry, converts the authorized JavaScript files from WebDAV into browser-memory **Blob URLs**, and mounts them into the UI context.

3. Security, Sandboxing, & Data Compliance

- **Strict Sandboxing:** To prevent malicious or poorly optimized code from compromising the central Yjs CRDT state or accessing decrypted **accounts.fsrsecure** credentials, all plugins with a visual interface are executed within highly restricted HTML5 **<iframe>** elements.
- **Communication Bridge (**postMessage**):** Plugins never possess direct access to the file system, the WebDAV client, or DOM network requests. Instead, they communicate with the core OS via the browser's native **postMessage** API. A plugin must transmit an "intent" (e.g., "Insert this table into the current document"). The OS core evaluates this request against the plugin's granted permissions before securely executing the Yjs transaction on its behalf.
- **Content Security Policy (CSP):** The OS enforces a strict CSP on the plugin execution context, neutralizing unauthorized external network requests to guarantee total data compliance and prevent data exfiltration.

Document: TechStack

1. Core Application & UI

- **Frontend Framework (SvelteKit):** I use SvelteKit as the foundational layer. It manages the overarching application architecture, routing, and UI state.
 - **Visual & Editing Layer (Tiptap):** Tiptap operates as the headless rich-text editor within the SvelteKit application. It provides high-performance rendering for complex documents, interactive maps, and visual boards, allowing for heavy customization of the editing experience without enforcing a specific UI.
-

2. Real-Time Collaboration & Synchronization

- **Conflict-Free State (Yjs):** Yjs serves as the real-time collaboration engine. It uses a CRDT (Conflict-free Replicated Data Type) algorithm to merge concurrent edits from multiple users automatically. The Tiptap editor's internal state is bound directly to a shared Yjs document, ensuring all clients stay perfectly synced without data loss or collision.
 - **Awareness Protocol:** The Yjs awareness module handles ephemeral, non-persistent state. This is what broadcasts live cursor positions, text selections, and "User X is typing..." indicators across the network.
 - **Network Transport:** WebSockets provide the real-time bridge, instantly broadcasting Yjs document updates and awareness data between the clients and the server.
-

3. Hybrid Storage Architecture I utilize a RAM-first, cloud-synced approach to balance real-time performance with secure persistence.

- **In-Memory Source of Truth:** A Node.js server operates as the central authority while a collaborative session is active. It holds the active Yjs documents in memory to process and broadcast state changes instantly.
 - **File-Based Persistence:** To save state, the Node.js server periodically flushes the in-memory Yjs data (either as a `.json` or a binary Yjs update file) directly to Sciebo via WebDAV. When a new session is initialized, the server pulls the latest file from Sciebo and loads it back into memory.
-

4. Multi-Tenancy & Infrastructure

- **Subdomain Isolation:** Each organization is assigned a unique subdomain (e.g., [fsr-design.fsr-os.de](#)).
- **Session Sandboxing:** Relying on subdomains ensures strict cookie and local storage isolation at the browser level, fundamentally preventing data leakage between different organizations.
- **Reverse Proxy:** Caddy (or Nginx) sits in front of the Node.js application to handle routing and automatically provision wildcard SSL certificates, ensuring secure HTTPS connections for every new tenant out of the box.

Document: Core File Structure

All files belonging to the FSR-OS have a custom file ending starting with .fsr...

1. File Naming Convention & Extensions

The system relies on a WebDAV backend (Sciebo) to store Yjs documents and system files.

ID-Only Naming (UI-Driven Naming)

- **Format:** `{id}.{extension}` (e.g., [1771675444568.fsrtasks](#))
 - **Mechanism:** The physical file on the WebDAV server is named purely by its internal ID. The human-readable filename is stored only inside the [meta.fsrsys](#) global index.
 - **Pros:** Highly efficient. Renaming a file in the OS only updates a text string inside the [meta.fsrsys](#) file. The physical file on WebDAV never moves or changes its name, eliminating [MOVE](#) commands and reducing sync conflicts.
 - **Cons:** The raw Sciebo WebDAV interface becomes unreadable, appearing as a list of numbers. Manual file recovery or management outside the OS UI becomes highly difficult.
 - **Mitigation ("Find on Server" Action):** To assist with native WebDAV management, the OS includes a "Find file on server" context action. This triggers a temporary WebDAV [MOVE](#) command, appending the current human-readable name to the file (`{name}-{id}.{extension}`) and simultaneously opening the exact Sciebo WebDAV folder URL in a new browser tab. A scheduled background job on the Node.js server automatically reverts the filename to its strict ID-only format after one hour to maintain system efficiency.
-

2. System Configuration Files (Admin-Accessible)

System configuration and operational data are separated into distinct file types based on their security requirements. These files are stored within protected directories using the `.sysfolder` extension. In the OS UI, these directories and their contents are exposed exclusively to accounts holding Admin roles.

- **Standard Configuration (`.fsrsys`):** Used for plain-text system rules and user preferences.
- **Encrypted Configuration (`.fsrsecure`):** Used for files containing sensitive Personally Identifiable Information (PII) or authentication tokens. To ensure data privacy, the backend automatically encrypts these files at rest (e.g., via AES-256) before writing them to the WebDAV storage. They are decrypted only within the server's memory during active use.

Fixed System Directories:

The core admin configuration is housed within the `config.sysfolder/` directory, which contains:

- `accounts.fsrsecure`: The globally encrypted identity registry mapping emails to internal Account IDs and hashed passwords.
 - `roles.fsrsys`: The global dictionary defining Role-Based Access Control (RBAC) permissions.
 - `accounts.sysfolder/`: A nested directory containing individual user profiles (e.g., `usr_123.fsrsys`), which store assigned roles, teams, and permanent UI preferences.
-

3. Hidden Infrastructure (OS Files and Folders)

Prefixing files or directories with a dot (.) ensures they are strictly invisible to *all* users in the OS UI, including Admins. These folders bypass the frontend completely and are managed exclusively by the Node.js backend under the hood to maintain application stability and performance.

Fixed Hidden Directories:

- `.system/`: The core operational directory. Crucially, this contains:
 - `meta.fsrsys` (or `meta.json` name is not fixed yet), the global directory index. This file maps all internal file IDs to their human-readable names and workspace paths, allowing the OS to render the file tree instantly without querying the WebDAV server. It is hidden to prevent accidental manual edits that would break system routing.

```
{
  "_meta": {
    "schemaVersion": "1.3",
    "lastUpdated": 1772988642243,
    "description": "Global index"
  },
  "nodes": {
    "fsr_d12a87731e34": {
      "name": "fsr_c5daf639f200",
      "parentId": null,
      "extension": ".workspace",
      "updatedAt": 1772986091000,
      "createdAt": 1772986092221,
      "tags": [],
      "customFields": {
        "isTemplate": false
      }
    },
    "fsr_18a287a8f1f3": {
      "name": "fsr_9f9b53660c27",
      "parentId": "fsr_d12a87731e34",
      "extension": "",
      "updatedAt": 1772986088000,
      "createdAt": 1772986092222,
      "tags": [],
      "customFields": {
        "isTemplate": false
      }
    },
    "fsr_5ec95bfd6774": {
      "name": "filenameder",
      "parentId": "fsr_18a287a8f1f3",
      "extension": ".fsr",
      "updatedAt": 1771681264000,
      "createdAt": 1772986092222,
      "tags": [],
      "customFields": {
        "isTemplate": false
      }
    },
    "fsr_8fa073978be0": {
      "name": "fsr_36779157993c",
      "parentId": "fsr_d12a87731e34",
      "extension": "",
      "updatedAt": 1772986088000,
```



```
"createdAt": 1772986092222,
"tags": [],
"customFields": {
  "isTemplate": false
}
},
"fsr_36df9eac9df2": {
  "name": "fsr_17b254fc369f",
  "parentId": "fsr_8fa073978be0",
  "extension": ".fsr",
  "updatedAt": 1771712358000,
  "createdAt": 1772986092222,
  "tags": [],
  "customFields": {
    "isTemplate": false
  }
},
"fsr_677859f85233": {
  "name": "fsr_b339773df18e",
  "parentId": "fsr_8fa073978be0",
  "extension": ".fsr",
  "updatedAt": 1771681280000,
  "createdAt": 1772986092222,
  "tags": [],
  "customFields": {
    "isTemplate": false
  }
},
"fsr_7e50a669bfd8": {
  "name": "filessd",
  "parentId": "fsr_8fa073978be0",
  "extension": ".fsr",
  "updatedAt": 1771675370000,
  "createdAt": 1772986092222,
  "tags": [],
  "customFields": {
    "isTemplate": false
  }
},
"fsr_9fc068dba4c2": {
  "name": "fsr_82032d56d63c",
  "parentId": "fsr_d12a87731e34",
  "extension": ".workspace",
  "updatedAt": 1772986087000,
  "createdAt": 1772986092222,
```

```
"tags": [],
"customFields": {
  "isTemplate": false
}
},
"fsr_4e1bdb8f92fb": {
  "name": "fsr_8401ff2be32f",
  "parentId": "fsr_9fc068dba4c2",
  "extension": ".fsr",
  "updatedAt": 1771763866000,
  "createdAt": 1772986092222,
  "tags": [],
  "customFields": {
    "isTemplate": false
  }
},
"fsr_ccfb848fbdea": {
  "name": "fsr_ae1b4af3d7f6",
  "parentId": "fsr_9fc068dba4c2",
  "extension": ".fsr",
  "updatedAt": 1771764488000,
  "createdAt": 1772986092222,
  "tags": [],
  "customFields": {
    "isTemplate": false
  }
},
"fsr_bdba4730f384": {
  "name": "fsr_398a67179471",
  "parentId": "fsr_d12a87731e34",
  "extension": ".fsr",
  "updatedAt": 1771712350000,
  "createdAt": 1772986092222,
  "tags": [],
  "customFields": {
    "isTemplate": false
  }
},
"fsr_c7a6ad0c0065": {
  "name": "file",
  "parentId": "fsr_d12a87731e34",
  "extension": ".fsrdoc",
  "updatedAt": 1772975246000,
  "createdAt": 1772986092222,
  "tags": [],
```

```

    "customFields": {
      "isTemplate": false
    }
  },
  "fsr_d597d09f3e57": {
    "name": "fsr_f533a6d42f67",
    "parentId": "fsr_d12a87731e34",
    "extension": ".jpg",
    "updatedAt": 1771596742000,
    "createdAt": 1772986092222,
    "tags": [],
    "customFields": {
      "isTemplate": false
    }
  },
  "fsr_ed112bf8edbb": {
    "name": "fsr_b7a58ad7b12a",
    "parentId": "fsr_d12a87731e34",
    "extension": ".fsr",
    "updatedAt": 1771795102000,
    "createdAt": 1772986092222,
    "tags": [],
    "customFields": {
      "isTemplate": false
    }
  }
}
}
}

```

- **.assets/**: The central storage repository for all raw binary files (images, PDFs) uploaded and embedded within **.fsrdoc** or **.fsrtask** files.
- **.trash/**: A hidden directory acting as a recycle bin. Deleted files are moved here, retaining their original ID and extension, allowing for easy restoration before a permanent cron-job purge.
- **.history/**: The storage location for Yjs document snapshots (version history). Storing these separately from the live documents prevents the active files from bloating system memory.
- **.locks/**: A temporary directory for zero-byte lock files. These are created during intensive WebDAV sync operations to prevent race conditions if multiple server instances attempt to write to infrastructure files simultaneously.

4. Folder Typology & Access Control

Separating the concept of "hidden system folders" from "admin-only configuration folders" provides a much cleaner architecture.

- **System Hidden Folders** (. prefix): * Examples: `.trash`, `.assets`, `.history`.
 - Behavior: These are strictly infrastructure. They should be completely hidden from the standard OS UI for *all* users, including admins. The OS interacts with them automatically under the hood (e.g., fetching an image from `.assets` when rendering a document).
 - **Admin-Only Folders** (`.fsrsysfolder`):
 - Examples: `configuration.sysfolder`, `security.sysfolder`.
 - Behavior: Visible and accessible only to accounts with the admin role. This is where you store your global configuration files like `accounts.fsrsys` and `roles.fsrsys`.
 - **Workspaces** (`.fsrworkspace`):
 - Behavior: Treated purely as a directory on the Sciebo/WebDAV backend, but the SvelteKit frontend intercepts the `.workspace` extension to render a distinct UI (e.g., a specific dashboard layout, custom icons, or forced Kanban views). Allowing them to be nested anywhere provides high flexibility without complicating the storage logic.
-

5. Template Files & Instantiation

To streamline repetitive workflows (e.g., standard meeting minutes, event checklists), any existing file can be converted into a reusable Template.

- **Conversion & UI:** Users with appropriate edit permissions can toggle a file's state to "Template" via its settings menu. Within the OS UI, templates are visually distinguished by a dotted border and specific iconography.
 - **Instantiation:** Clicking a template does not open it for direct editing. Instead, it prompts an instantiation modal ("Create file from template?"). Upon confirmation, the backend duplicates the file's exact state, structure, and rich-text content into a new, independent file within the current workspace.
 - **Architecture:** Templates do not use unique file extensions. They retain their native format (e.g., `.fsrdoc` or `.fsrtask`) but are flagged with an `isTemplate: true` boolean within the global `meta.fsrsys` directory index. The duplication process relies on a native WebDAV `COPY` command, ensuring fast and exact replication of the underlying CRDT data without bogging down the Node.js memory. To modify the core template itself, users access a specific "Edit Template" route via the file's context menu.
-

Handling Standard Files (Images, PDFs, and External Assets)

While the system primarily operates on native collaborative file types (e.g., `.fsrdoc`, `.fsrtasks`), it fully supports standard binary files (such as `.jpg`, `.png`, `.pdf`, or `.zip`) uploaded either through the OS interface or placed directly into the storage folder via the native Sciebo client.

1. Detection & Indexing

To ensure standard files appear instantly in the OS directory tree without degrading performance through constant WebDAV polling, the Node.js backend utilizes a background sync process. When an unindexed file (e.g., `Brand Guidelines.pdf`) is detected in a tracked workspace, the system automatically generates an internal tracking ID with an `ext_` prefix. It then injects this file into the central `meta.frsys` index with the property `type: "external"`. Crucially, the file's physical name remains completely untouched on the WebDAV server, preserving native readability.

2. Storage & Embedding Standard files exist in two primary contexts:

- **Workspace Files:** Uploaded directly into a visible workspace folder alongside `.fsrdoc` files. These are indexed globally and visible in the standard directory tree.
- **Embedded Assets:** Images or PDFs uploaded specifically to be embedded *inside* a rich text document or a Kanban card. These are routed directly to the hidden `.assets/` system folder to keep the main workspaces visually uncluttered, while still being indexed in `meta.frsys` for fast retrieval.

3. Rendering & Previews

When a user opens an external file within the OS, the frontend identifies the `type: "external"` flag and bypasses the Yjs CRDT collaboration engine. Based on the file extension, the OS triggers a dedicated preview handler:

- **Images (`.jpg`, `.png`, `.svg`):** The Node.js server securely proxies the binary stream from WebDAV to the SvelteKit frontend. The OS renders the payload directly within an image viewer component, supporting zoom and pan interactions.
- **Documents (`.pdf`):** The binary stream is piped into a native browser `<iframe>` or a dedicated PDF rendering wrapper, allowing users to scroll and read the document inline without needing to download it locally.

- **Unsupported Formats:** If a file format cannot be rendered natively in the browser (e.g., `.docx` or `.zip`), the OS displays a generic file icon card with metadata (file size, upload date) and a prominent "Download" button to retrieve the file from Sciebo.

Document: Accounts Management and User Preferences

Overview

Identity, Access, and State Management The architecture enforces a strict separation between highly sensitive login credentials, global permission definitions, and individual user profiles. This prevents write-bottlenecks on the WebDAV storage server and ensures that login data is isolated and encrypted at rest. The system avoids permanent WebDAV file locks; instead, the Node.js backend operates via a master service account with sole write-access to the raw storage, while users interact with the files safely through the OS interface.

1. Global Identity Registry

(`config.sysfolder/accounts.fsrsecure`)

- **Function:** The central, highly secure lookup directory for authentication.
- **Data Structure:** Contains a minimal mapping of Email Addresses to `account-ids` and hashed credentials (or SSO provider tokens).
- **Security:** Utilizing the `.fsrsecure` extension, this file is symmetrically encrypted at rest (e.g., AES-256-GCM) by the backend before being saved to the storage server. It is decrypted entirely in memory during the authentication handshake purely to verify logins and resolve the `account-id`. It is never exposed to the frontend OS.

Example JSON structure:

```
{
  "_meta": {
    "schemaVersion": "1.0",
    "lastUpdated": 1771678000000,
    "description": "Encrypted identity registry. Maps login methods to internal account IDs."
  },
  "identities": {
```

```

"admin@fsr-design.de": {
  "accountId": "usr_1771675444568",
  "authType": "local",
  "passwordHash": "$argon2id$v=19$m=65536,t=3,p=4$qWeRtYuloP...",
  "isActive": true,
  "securityFlags": {
    "requiresPasswordReset": false,
    "mfaEnabled": true,
    "mfaSecret": "JBSWY3DPEHPK3PXP"
  }
},
"freelancer@example.com": {
  "accountId": "usr_1771680123456",
  "authType": "sso_google",
  "ssoProviderId": "1048593029485039485",
  "isActive": true,
  "securityFlags": {
    "requiresPasswordReset": false,
    "mfaEnabled": false
  }
},
"client.viewer@company.com": {
  "accountId": "usr_1771695555555",
  "authType": "magic_link",
  "magicLinkTokenHash": "a1b2c3d4e5f6g7h8i9j0...",
  "tokenExpiresAt": 1771700000000,
  "isActive": true,
  "securityFlags": {}
}
}
}

```

2. Role & Permission Definitions

([config.sysfolder/roles.frsys](#))

- **Function:** The global dictionary that defines what specific roles can actually do.
- **Data Structure:** Contains the raw permission flags assigned to role strings (e.g., defining that the "Editor" role has [can_edit_files: true](#) and [can_delete_workspaces: false](#)).
- **Implementation:** This file contains no user data or assignments. It is stored in plain text, allowing the backend to cache the permission rules in memory without decrypting sensitive information.

more detailed permission and role overview:

[Role & Permission Matrix](#)

example structure:

```
{
  "_meta": {
    "schemaVersion": "1.1",
    "lastUpdated": 1771678500000,
    "description": "Global Role-Based Access Control (RBAC) definitions."
  },
  "roles": {
    "admin": {
      "inherits": "editor",
      "permissions": {
        "system.manage_users": true,
        "system.manage_roles": true,
        "system.view_sysfolders": true
      }
    },
    "editor": {
      "inherits": "guest",
      "permissions": {
        "workspace.create": true,
        "workspace.delete": true,
        "files.create": true,
        "files.edit": true,
        "files.delete": true,
        "files.move": true
      }
    },
    "finanzer": {
      "inherits": "guest",
      "permissions": {},
      "overrides": {
        "workspaces": {
          "ws_17716000000099": {
            "files.create": true,
            "files.edit": true
          }
        }
      }
    },
    "files": {
      "budget_2026-1771675444568.fsrtbl": {
        "files.edit": true,
        "files.delete": false
      }
    }
  }
}
```



```

    },
    "q1_invoices-1771675444600.fsrdoc": {
      "files.edit": true
    }
  }
},
"guest": {
  "inherits": null,
  "permissions": {
    "workspace.read": true,
    "files.read": true
  }
}
}
}

```

3. User Account & Preference Files

([config.sysfolder/accounts.sysfolder/{account-id}.fsrsys](#))

- **Function:** The comprehensive profile for an individual user, storing their system state and directory associations.
- **Data Structure:** Tied directly to the [account-id](#), this file stores the user's assigned roles (e.g., [role: "Editor"](#)), their team/department associations, and permanent UI preferences (default notification routing, language, pinned workspaces).
- **Implementation:** By isolating assignments and preferences into individual files, the system eliminates concurrent write conflicts. If an admin changes a user's team, or a user updates their dashboard layout, the server only syncs that specific [{account-id}.fsrsys](#) file back to WebDAV. Ephemeral UI states (e.g., dark mode) bypass this entirely and are managed via browser `LocalStorage`.

example structure for an account

([config.sysfolder/accounts.sysfolder/usr_1771675444568.fsrsys](#)):

```

{
  "_meta": {
    "schemaVersion": "1.0",
    "lastUpdated": 1771678050000,
    "accountId": "usr_1771675444568"
  },
  "profile": {
    "firstName": "Alex",
    "lastName": "Designer",

```

```

"displayname": "Alex D.",
"avatarurl": "/assets/avatars/usr_1771675444568_thumb.jpg",
"statusmessage": "Working on the new design system"
},
"access": {
  "roles": ["admin", "content_editor"],
  "teams": ["team_design", "team_management"],
  "workspaces": {
    "ws_1771600000000": "owner",
    "ws_1771600000001": "viewer"
  }
},
"preferences": {
  "language": "en-US",
  "timezone": "Europe/Berlin",
  "notifications": {
    "emailmentions": true,
    "emaildailydigest": false,
    "inappsound": true
  },
  "ui": {
    "pinnedworkspaces": ["ws_1771600000000", "ws_1771600000042"],
    "defaultstartupview": "dashboard"
  }
}
}

```

Document: Role Permission Matrix

1. Global Roles

The system relies on a hierarchical role structure defined in the [roles.frsys](#) configuration. Every user interacting with the platform is assigned a role that determines their baseline access level.

- **Admin:** Full system access. Can configure the BYOS (Bring Your Own Storage) Sciebo credentials, change global branding, manage user roles, and delete entire Spaces.
- **Member:** Standard FSR council member. Can create, edit, and delete files within the OS view. Can assign Team Labels to files and folders.
- **Guest / Helper:** Limited access. Interacts primarily through the Web Preview view. Can only perform specific, granular actions defined by the file

type, such as typing their name into an open Shift/Roster slot or buying a ticket. Cannot edit core file configurations or system settings.

- **Anonymous (Not Logged In):** Represents public users without an active account session. Crucially, the permissions for this baseline role are fully adjustable within the system. Depending on the setup, anonymous users can be granted access to view specific public files, submit form responses, or interact with open event rosters without needing to create an account.

2. File-Level & Space-Level Permissions

Beyond global roles, access is filtered down to the specific file and folder (Space) level.

- **Public vs. Private:** Any file can be toggled to "Public." If public, it is rendered on the external Web Preview site (accessible to roles like Guest or Anonymous, depending on settings). If private, it only exists and is accessible within the internal OS view.
 - **Space Restrictions:** Entire directories (Spaces) can be locked to specific Team Labels. For example, a workspace named `space-finanzen` can be restricted so that only users who possess the "Finance Team" label in their profile can open or view its contents.
-

System Permissions

- **Function:** The global dictionary that defines what specific roles can actually do.
- **Data Structure:** Contains the raw permission flags assigned to role strings (e.g., defining that the "Editor" role has `can_edit_files: true` and `can_delete_workspaces: false`).
- **Implementation:** This file contains no user data or assignments. It is stored in plain text, allowing the backend to cache the permission rules in memory without decrypting sensitive information.

These flags control access to global settings, identity management, and underlying infrastructure.

- `system.manage_users`: Allows inviting new users, suspending accounts, and modifying assignments in the `accounts.sysfolder`.
- `system.manage_roles`: Grants write access to the `roles.frsys` dictionary to create or modify global permission profiles.
- `system.view_sysfolders`: Allows the OS UI to render and open `.sysfolder` directories (e.g., `config.sysfolder`). Without this, these directories remain completely invisible.

- **system.settings.write**: Allows modifying global variables, such as BYOS (Sciebo) WebDAV credentials, system-wide branding, and custom domains.

Workspace Permissions (**workspace.***)

These flags govern the structural organization of the OS layer.

- **workspace.create**: Allows creating new top-level workspaces or sub-folders.
- **workspace.edit**: Allows renaming existing workspaces or changing their associated metadata.
- **workspace.delete**: Allows moving an entire workspace (and its contents) to the **.trash** directory.
- **workspace.manage_access**: Allows locking or unlocking workspaces to specific Team Labels (e.g., restricting a folder to the "Finance Team").

File Permissions (**files.***)

These flags dictate how users interact with the actual Yjs CRDT documents and WebDAV assets.

- **files.read**: The baseline requirement to open a file in the OS and view its contents.
- **files.create**: Allows generating new **.fsrdoc**, **.fsrtask**, or other standard file types within an accessible workspace.
- **files.edit**: Allows modifying the internal data of a file (e.g., typing in a document, dragging a Kanban card).
- **files.delete**: Allows moving individual files to the **.trash** directory.
- **files.move**: Allows changing a file's parent directory/workspace ID.
- **files.publish**: Allows toggling a file's status to "Public," making it accessible via the external Web Preview routing.

Interaction Permissions (**interaction.***)

These flags are highly granular and typically assigned to the Anonymous or Guest roles to allow controlled interaction without full edit rights.

- **interaction.submit_forms**: Allows appending data to a **.fsrform** file (submitting a survey) without seeing other users' submissions.
- **interaction.book_shifts**: Allows adding a name to an open slot in a **.fsrrost** (Roster) file without the ability to modify the shift times or delete other users.
- **interaction.checkout**: Allows interacting with Stripe integration blocks inside files to purchase tickets or merchandise.

Document: Feature Explanation

1. Global Search & Command Palette (Cmd+K)

- **Function:** A centralized, keyboard-driven interface to navigate the OS, execute actions, and find data instantly without using the mouse.
 - **Architecture:** The Command Palette primarily queries the in-memory `meta.frsys` index. Because the index is a flat JSON structure loaded entirely in the frontend, searching for file names, tags, or workspaces happens with zero network latency.
 - **Deep Content Search:** Searching for specific text *inside* a document requires backend processing. The Node.js server maintains a lightweight search index (e.g., using Lunr.js or SQLite FTS) that updates asynchronously when files are saved to WebDAV, returning deep-search results to the Command Palette without blocking the main thread.
-

2. Cross-File Variables (The `$System`)

- **Function:** Allows users to define a single source of truth for dynamic data (e.g., `$jahresbudget`, `$event_date`) in one file and seamlessly embed it in any other file across the workspace.
- **Declaration:** Users can highlight text in a document, or select a cell in a spreadsheet, and declare it as a global variable. The UI then assigns it a unique string identifier (e.g., `$budget`).
- **Implementation:** Variables are extracted from their native Yjs documents and mirrored into a global `variables` dictionary within the `meta.frsys` index.
- **Rendering:** When a user types `$budget` into a Tiptap document, a custom Svelte component intercepts the string. It queries the local `meta.frsys` store and renders the current value as a reactive, read-only pill. If the source value is modified in the original spreadsheet, the `meta.frsys` store updates, instantly pushing the new value to all documents currently displaying that variable via WebSocket.

The "Ghost Calculation" Problem

Because the system uses Yjs and client-side rendering (SvelteKit/Canvas), the actual logic that calculates `A2 + A4` or `$startmoney - $expenses` runs in the user's browser. The Node.js server only stores the resulting data state.

If a user has **File A** open and changes `A2`, the browser calculates the new `$startmoney` and pushes that updated value to `meta.frsys`. Any open file currently displaying `$startmoney` updates instantly.

However, if **File B** (which calculates `$currentmoney` based on `$startmoney`) is currently *closed* (no user has it open in a browser), there is no engine running to calculate the new result. **File C** will continue to display the old, stale value for `$currentmoney` because `meta.fsrsys` was never told that File B's output changed.

To solve this, there are three architectural paths.

Option 1: The Headless Server Engine (Too Complex for Phase 1)

The Node.js server would need to maintain a strict dependency graph. When `$startmoney` updates in `meta.fsrsys`, the server notices that File B depends on it. The server would have to download File B from Sciebo into a headless, invisible Yjs instance, run the spreadsheet logic to calculate the new `$currentmoney`, update `meta.fsrsys` again, and then broadcast the final value to File C.

- **Verdict:** Highly resource-intensive. For a sole developer, building a headless server-side spreadsheet calculation engine will derail the MVP timeline.

Option 2: The "Dirty Flag" & Lazy Evaluation (The Compromise)

`meta.fsrsys` tracks which variables depend on others. When `$startmoney` changes, the server simply flags `$currentmoney` in `meta.fsrsys` as `status: "stale"`. If a user has File C open, the `$currentmoney` pill turns yellow or displays a warning icon. It tells the user: *"This value is outdated because the source file (\$startmoney) changed. Please open File B to recalculate."* As soon as any user opens File B, their browser automatically runs the math, pushes the fresh `$currentmoney` to `meta.fsrsys`, and File C turns green and updates.

- **Verdict:** Transparent and honest to the user, but requires managing dependency maps in the JSON file.

Option 3: Prevent Variable Nesting (The Pragmatic MVP Choice)

The cleanest way to ensure data integrity for the initial release is to enforce a strict rule: **Global variables can only be defined from raw file outputs, not from formulas containing other global variables.** You can define `$startmoney` in a spreadsheet. You can display it in a document. But the UI will disable the "declare as variable" button on a spreadsheet cell if that cell's formula includes an external `$variable`.

- **Verdict:** This completely eliminates the Ghost Calculation problem. It guarantees that `meta.fsrsys` is always 100% accurate because a variable never secretly relies on closed files to compute.

Document: File Types

Overview

FSR-OS Files are a specialized JSON file storing all the data within them. Before a websocket is closed the data is stored through Yjs within the RAM of the server but after that it is deployed to the original json, stored on the connected cloud

Each FSR-OS file has its own file extension starting with fsr like .fsrdoc

Phase 1 Filetypes:

1. **Event Items:** The core configuration metadata for an event. **(.fsrevt)**
2. **Documents:** Rich text protocols (Markdown/Tiptap) with custom variables. **(.fsrdoc)**
3. **Tasks (Kanban/List):** Real-time board for project management. **(.fsrtasks)**
4. **Tables / Spreadsheets:** Grid-based data management. **(.fsrtable)**
5. **Roster / Shift:** Sign-up sheets with optimistic UI updates for helpers. **(.fsrroster)**

other Filetypes:

1. **Stripe Checkout File:** Integrated ticketing and payment forms. **(.fsrcheckout)**
 2. Chats: a File that acts like a Whatsapp Chat **(.fsrchat)**
 3. Survey / Form File **(.fsrform)**
 4. maybe a "View" File Type where you can List only specific tagged things, like in Linear **(.fsrview)**
-

In Depth

1. Event Items (Configuration Metadata) - .fsrevt



- **Features:** Centralized settings, dates, locations, and global variables defining the event, collecting event related Files and sub events
- **Viability:** High / Straightforward.
- **Technology:** I will use a **Y.Map** to store fundamental key-value pairs (e.g., **title: string, date: timestamp**). SvelteKit will render standard form inputs. Two-way binding ensures that when a value changes in the UI, the **Y.Map** updates, instantly syncing the event metadata to all connected clients.

2. Documents (Rich Text Protocols) - .fsrdoc



- **Features:** Collaborative text editing with formatting, embedding, and dynamic variables (e.g., pulling the event name directly from the Event Item). Add special elements like tables or file links with "/" command. Textedit tools will be a bar that appears when text is highlighted like in Linear and Notion
- **Viability:** High / Core functionality.
- **Technology:** Tiptap serves as the editor interface, bound to a `Y.XmlFragment` via the `tiptap-extension-collaboration` package. To handle custom variables, I will create a custom Tiptap Node that queries the loaded Event Item `Y.Map` and renders its current value dynamically within the text flow.

3. Tasks (Kanban / List) - .fsrtasks



- **Features:** Columns, drag-and-drop task cards, assignees, and real-time status updates.
- **Viability:** High.
- **Technology:** The data model relies on a [Y.Map](#) for the board state. Keys are column IDs, and values are [Y.Arrays](#) containing task objects (or task IDs). When I drag a card in the SvelteKit UI (using a library like [svelte-dnd-action](#)), the local client splices the task out of the source [Y.Array](#) and pushes it into the destination [Y.Array](#). Yjs automatically calculates the delta and broadcasts the exact movement.

4. Tables / Spreadsheets - `.fsrtable`



- **Features:** Grid-based data management, dynamic rows/columns, and cell editing.
- **Viability:** Medium-High (UI complexity is higher than data complexity).
- **Technology:** There are two approaches. I can either use Tiptap's native Table extension (which maps a table structure into a `Y.XmlFragment`) for document-embedded tables, or build a standalone grid component in SvelteKit backed by a `Y.Array` of `Y.Map` objects (representing rows). The latter offers more flexibility for strict data typing and spreadsheet-like calculations.

5. Roster / Shift (Sign-up Sheets) - `.fsrroster`

- **Features:** Defined time slots, role requirements, and fast sign-up toggles for helpers. If users aren't logged in it asks for a name
- **Viability:** High.
- **Technology:** A `Y.Array` holds the individual shift objects. Each shift object contains a nested `Y.Array` for `assignedUsers`. Because Yjs applies mutations locally before broadcasting, clicking "Sign Up" pushes the user ID to the array instantly, creating a native optimistic UI update without writing complex asynchronous rollback logic.

6. Stripe Checkout File - `.fsrcheckout`

- **Features:** Integrated payment forms, ticket pricing configuration, and inventory limits.
- **Viability:** Medium (Requires secure server-side coordination).

- **Technology:** A `Y.Map` stores the public configuration (ticket tiers, prices). However, payment processing cannot rely solely on client-side CRDTs. The SvelteKit UI will trigger a server action on the Node.js backend to generate a secure Stripe Checkout session. Stripe webhooks will then update a protected server-side database (or a secure, read-only section of a Yjs document) to reflect successful payments and decrement available stock.

7. Chats - .fsrchat

- **Features:** WhatsApp-style message logs, typing indicators, and read receipts.
- **Viability:** High.
- **Technology:** An append-only `Y.Array` stores message objects. The Yjs Awareness protocol manages the ephemeral "User is typing..." state via WebSockets without polluting the persistent document storage.

8. Survey / Form File - .fsrform

- **Features:** Drag-and-drop builder for questions, public sharing links, and submission collection.
- **Viability:** High (Separation of builder and responses is key).
- **Technology:** The form builder uses a `Y.Array` of schema objects (defining the questions). For submissions, a standard REST endpoint or server action is preferable. The Node.js server receives the submission and pushes it into a private `Y.Array` (or separate database) so that anonymous users filling out the form do not receive the synchronized state of everyone else's private answers.

9. "View" File (Linear-style Aggregation) - .fsrview

- **Features:** Read-only or proxy-editable lists filtering tasks or items across multiple different files based on specific tags (e.g., showing all "urgent" tasks from all Kanban boards).
- **Viability:** Medium-High (Requires cross-document querying).
- **Technology:** The "View" file itself is just a `Y.Map` storing query parameters (e.g., `filters: ['urgent']`). The challenge is data retrieval. The SvelteKit client (or Node.js server) must iterate through the metadata of other available Yjs documents in the workspace, aggregate the items matching the tags, and render a unified list.

Document: Organization Setup Flow

Provisioning a New Organization (Phase 1 & 2)

When a new FSR is onboarded, the system follows this automated sequence to maintain data isolation:

1. **Subdomain Routing:** The organization is assigned a unique subdomain (e.g., fsr07.fsr-os.de).
2. **Tenant Registration:** The Node.js server reads the subdomain and registers a new [tenant_id](#) in a local SQLite database on the server.
3. **BYOS Credential Link:** The FSR Admin inputs their Sciebo WebDAV URL, Username, and App Password. These are encrypted and stored in the local SQLite database linked to the [tenant_id](#).
4. **Directory Scaffolding:** Upon first successful connection, the server pushes a default folder structure (e.g., [Assets](#), [Events](#), [Templates](#)) and a master [meta.frsys](#) file to the Sciebo cloud.
5. **Branding Setup:** The Admin uploads a logo and selects primary/secondary colors, which are saved to the [meta.frsys](#) to style the Web Preview frontend.

FSR-OS Privacy & Security Architecture

1. Concern: Exposure of Cloud Storage Passwords

The Risk: Because the server acts as a middleman for real-time collaboration, it requires access to the organization's cloud storage (WebDAV/Sciebo), which raises concerns about rogue access to university files.

Current Solution (App Passwords): FSR-OS strictly mandates the use of revocable "App Passwords" (Device Passwords) rather than primary university credentials. This gives the organization an ultimate "kill switch" to revoke the app's access instantly from their Sciebo dashboard.

Target Solution (OAuth 2.0): Transitioning to an OAuth 2.0 flow. This ensures the application never handles user passwords at all, relying entirely on mathematically secure, revocable access tokens.

2. Concern: Secret Logging or Data Hoarding

The Risk: Users may worry that the server secretly logs sensitive files (like [accounts.fsrsecure](#)) to a hidden hard drive while processing them in RAM.

The Solution (Stateless, Read-Only Infrastructure): The application is deployed via Docker containers with a Read-Only root filesystem. The operating system physically blocks the Node.js application from writing any files to the disk. The

only writable location is a strictly defined volume mount dedicated exclusively to the SQLite tenant database. Network streaming (uploading/downloading to WebDAV) happens entirely in memory without requiring disk writes.

3. Concern: Data Exposure via In-Memory Caching

The Risk: To maintain high performance, the system caches cloud data in RAM. Keeping highly sensitive files in RAM for too long increases the attack surface if the server is compromised.

The Solution (Data Partitioning): Caching logic is partitioned based on data sensitivity. The structural file index (meta.fsrsys) is cached for 15 minutes to ensure high-speed UI navigation. However, highly sensitive configuration or account files (e.g., accounts.fsrsecure) bypass the cache or are cached for an ultra-short TTL (e.g., 1 minute). Furthermore, all cached data lives in volatile RAM that vanishes permanently upon server restart.

4. Concern: Total Control & Hosting Trust

The Risk: Strict university IT departments may unconditionally refuse to route their data through a third-party hosted SaaS environment.

The Solution (Self-Hosting Escape Hatch): Because FSR-OS is a lean stack (Node.js + SQLite), it is packaged into a standalone Docker Image. Organizations that require absolute data sovereignty can deploy this image on their own internal university servers, effectively using the software without relying on our hosted infrastructure.

5. Concern: Code Transparency vs. Intellectual Property Protection

The Risk: Proving the code is not malicious usually requires making it Open Source, which allows competitors to fork and steal the intellectual property.

The Solution (Source-Available Audits & Compiled Distribution): The repository remains closed-source. To build trust, strict IT departments can be granted temporary, Read-Only access to the codebase under a Non-Disclosure Agreement (NDA) for security audits. Additionally, self-hosted Docker images contain only compiled, minified code (or bundled binaries), functioning as a secure "black box" that protects the raw source code from being modified or easily reverse-engineered

